

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from factor_analyzer.factor_analyzer import calculate_bartlett_sphericity, calcu

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_predict, train_te
from sklearn.metrics import roc_auc_score, accuracy_score, silhouette_score, cal
from sklearn.cluster import KMeans

from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.stats import kruskal, chi2_contingency

import warnings

warnings.filterwarnings('ignore')
```

Análisis multivariante sobre los tipos de consumo

Análisis de Componentes Principales

Primero vamos a hacer el acp básico, cogiendo las componentes que expliquen más de un 75% de la variabilidad para imputar la variables sector público, y posteriormente haremos el análisis definitivo.

```
In [2]: datos = pd.read_csv('../datos_procesados/datos_iniciales.csv')
proporciones = [col for col in datos.columns if col.startswith('prop_')]

seed = 111104

datos_train, datos_test = train_test_split(datos, test_size=0.3, random_state=se

print(f"Datos de entrenamiento: {datos_train.shape[0]} hogares")
print(f"Datos de prueba: {datos_test.shape[0]} hogares")
```

Datos de entrenamiento: 14493 hogares
 Datos de prueba: 6212 hogares

```
In [3]: datos_prop_train = datos_train[proporciones]
datos_prop_test = datos_test[proporciones]

print("Dimensiones de datos_prop_train:", datos_prop_train.shape)
print("Dimensiones de datos_prop_test:", datos_prop_test.shape)
```

Dimensiones de datos_prop_train: (14493, 12)
 Dimensiones de datos_prop_test: (6212, 12)

```
In [4]: chi_square, p_value = calculate_bartlett_sphericity(datos_prop_train)
```

```
kmo_all, kmo_model = calculate_kmo(datos_prop_train)

print(f"Chi-cuadrado de Bartlett: {chi_square}")
print(f"Valor p: {p_value}")
print(f"Medida KMO global: {kmo_model.round(4)}")
```

Chi-cuadrado de Bartlett: nan
 Valor p: nan
 Medida KMO global: 0.2775

Imputación de sector público

```
In [5]: print("Proporción de valores nulos en 'sector_publico' en datos_train:")
print(sum(datos_train["sector_publico"].isna())/datos_train.shape[0])
print("Proporción de valores nulos en 'sector_publico' en datos_test:")
print(sum(datos_test["sector_publico"].isna())/datos_test.shape[0])
print("Proporción de valores nulos en 'sector_publico' en datos:")
print(sum(datos["sector_publico"].isna())/datos.shape[0])
```

Proporción de valores nulos en 'sector_publico' en datos_train:
 0.09701235079003658
 Proporción de valores nulos en 'sector_publico' en datos_test:
 0.09867997424339987
 Proporción de valores nulos en 'sector_publico' en datos:
 0.097512678097078

```
In [6]: escalador_imp = StandardScaler()
datos_prop_train_escalados = escalador_imp.fit_transform(datos_prop_train)
pca_imp = PCA()
pca_imp.fit_transform(datos_prop_train_escalados)

tabla_acp = pd.DataFrame({
    "Componente": [f"CP{i+1}" for i in range(len(proporciones))],
    "Autovalor": pca_imp.explained_variance_,
    "% varianza explicada": pca_imp.explained_variance_ratio_ * 100,
    "% varianza acumulada": np.cumsum(pca_imp.explained_variance_ratio_) * 100
})

display(tabla_acp)
```

	Componente	Autovalor	% varianza explicada	% varianza acumulada
0	CP1	1.847831e+00	1.539753e+01	15.397526
1	CP2	1.255109e+00	1.045852e+01	25.856048
2	CP3	1.177191e+00	9.809246e+00	35.665294
3	CP4	1.082373e+00	9.019149e+00	44.684443
4	CP5	1.023097e+00	8.525217e+00	53.209660
5	CP6	1.019625e+00	8.496289e+00	61.705949
6	CP7	9.729802e-01	8.107609e+00	69.813558
7	CP8	9.419981e-01	7.849443e+00	77.663001
8	CP9	9.394361e-01	7.828094e+00	85.491095
9	CP10	8.949294e-01	7.457230e+00	92.948325
10	CP11	8.462594e-01	7.051675e+00	100.000000
11	CP12	4.033232e-17	3.360795e-16	100.000000

Cogeremos 8 componentes para la imputación, + las variables edad, nivel de estudios y tamaño del municipio

```
In [7]: coordenadas_train = pca_imp.transform(datos_prop_train_escalados)
componentes_train_df = pd.DataFrame(
    coordenadas_train,
    columns=[f"CP{i+1}" for i in range(len(proporciones))],
    index=datos_train.index
)

datos_prop_test_escalados_imp = escalador_imp.transform(datos_prop_test)
coordenadas_test = pca_imp.transform(datos_prop_test_escalados_imp)
componentes_test_df = pd.DataFrame(
    coordenadas_test,
    columns=[f"CP{i+1}" for i in range(len(proporciones))],
    index=datos_test.index
)

for i in range(1, 9):
    datos_train[f"CP{i}"] = componentes_train_df[f"CP{i}"]
    datos_test[f"CP{i}"] = componentes_test_df[f"CP{i}"]

variables_componentes = [f"CP{i}" for i in range(1, 9)]
variables_sociodemograficas = ['edad', 'estudios', 'tam_municipio']

datos_sociodemograficos_dummies_train = pd.get_dummies(datos_train[variables_soc
datos_sociodemograficos_dummies_test = pd.get_dummies(datos_test[variables_socio
datos_sociodemograficos_dummies_test = datos_sociodemograficos_dummies_test.rein

X_total_train = pd.concat([datos_train[variables_componentes], datos_sociodemogr
X_total_test = pd.concat([datos_test[variables_componentes], datos_sociodemograp

datos_obs_train = datos_train[datos_train["sector_publico"].notna()].copy()
X_obs_train = X_total_train.loc[datos_obs_train.index]
y_obs_train = datos_obs_train["sector_publico"].astype(int)
```

```

datos_train["sector_publico_imp"] = datos_train["sector_publico"]
datos_test["sector_publico_imp"] = datos_test["sector_publico"]

datos_mis_train = datos_train[datos_train["sector_publico"].isna()]
datos_mis_test = datos_test[datos_test["sector_publico"].isna()]

X_mis_train = X_total_train.loc[datos_mis_train.index]
X_mis_test = X_total_test.loc[datos_mis_test.index]

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=111104
)

modelo = LogisticRegression(max_iter=2000, random_state=111104)

resultados = []

prob_cv = cross_val_predict(
    modelo,
    X_obs_train,
    y_obs_train,
    cv=cv,
    method="predict_proba"
)[: , 1]

pred_cv = (prob_cv >= 0.5).astype(int)

tabla_validacion_imp = pd.DataFrame({
    "AUC CV": roc_auc_score(y_obs_train, prob_cv),
    "Accuracy CV": accuracy_score(y_obs_train, pred_cv),
    "% predicho como sector_publico=1": pred_cv.mean() * 100
}, index=[0])

display(tabla_validacion_imp.round(4))

```

	AUC CV	Accuracy CV	% predicho como sector_publico=1
0	0.7233	0.7734	10.0481

```

In [8]: modelo_final_imputacion = LogisticRegression(max_iter=2000, random_state=111104)
modelo_final_imputacion.fit(X_obs_train, y_obs_train)

rng = np.random.default_rng(seed=111104)

if not X_mis_train.empty:
    prob_mis_train = modelo_final_imputacion.predict_proba(X_mis_train)[: , 1]
    datos_train.loc[datos_train["sector_publico"].isna(), "sector_publico_imp"]

if not X_mis_test.empty:
    prob_mis_test = modelo_final_imputacion.predict_proba(X_mis_test)[: , 1]
    datos_test.loc[datos_test["sector_publico"].isna(), "sector_publico_imp"] =

datos_train["sector_publico_imp"] = datos_train["sector_publico_imp"].astype(int)
datos_test["sector_publico_imp"] = datos_test["sector_publico_imp"].astype(int)

print("Missing en sector_publico_imp:")

```

```

print(datos_train["sector_publico_imp"].isna().sum())

print("\nDistribución observada original:")
print(datos_train["sector_publico"].value_counts(normalize=True, dropna=True))

print("\nDistribución final tras imputación:")
print(datos_train["sector_publico_imp"].value_counts(normalize=True))

```

Missing en sector_publico_imp:
0

Distribución observada original:
sector_publico
0.0 0.764193
1.0 0.235807
Name: proportion, dtype: float64

Distribución final tras imputación:
sector_publico_imp
0 0.764162
1 0.235838
Name: proportion, dtype: float64

```

In [9]: datos_train['particion'] = 'train'
datos_test['particion'] = 'test'
datos = pd.concat([datos_train, datos_test], axis=0)

for i in range(1, 9):
    if f"CP{i}" in datos.columns:
        del(datos[f"CP{i}"])

```

ACP principal

```

In [10]: escalador = StandardScaler()

datos_prop_train_escalados = escalador.fit_transform(datos_prop_train)
datos_prop_test_escalados = escalador.transform(datos_prop_test)
pca = PCA()
pca.fit_transform(datos_prop_train_escalados)

tabla_acp = pd.DataFrame({
    "Componente": [f"CP{i+1}" for i in range(len(proporciones))],
    "Autovalor": pca.explained_variance_,
    "% varianza explicada": pca.explained_variance_ratio_ * 100,
    "% varianza acumulada": np.cumsum(pca.explained_variance_ratio_) * 100
})

autovalores = tabla_acp["Autovalor"]
prop_varianza = tabla_acp["% varianza explicada"]/100
prop_acumulada = tabla_acp["% varianza acumulada"]/100

p = len(proporciones)

display(tabla_acp.round(5))

```

	Componente	Autovalor	% varianza explicada	% varianza acumulada
0	CP1	1.84783	15.39753	15.39753
1	CP2	1.25511	10.45852	25.85605
2	CP3	1.17719	9.80925	35.66529
3	CP4	1.08237	9.01915	44.68444
4	CP5	1.02310	8.52522	53.20966
5	CP6	1.01963	8.49629	61.70595
6	CP7	0.97298	8.10761	69.81356
7	CP8	0.94200	7.84944	77.66300
8	CP9	0.93944	7.82809	85.49109
9	CP10	0.89493	7.45723	92.94833
10	CP11	0.84626	7.05167	100.00000
11	CP12	0.00000	0.00000	100.00000

La 12ª componente nos sale sin autovalor y sin varianza explicada, debido a la composición de los datos, ya que todas las variables de proporciones suman 1.

```
In [11]: m_65 = np.argmax(prop_acumulada >= 0.65) + 1
m_70 = np.argmax(prop_acumulada >= 0.70) + 1
m_75 = np.argmax(prop_acumulada >= 0.75) + 1
m_80 = np.argmax(prop_acumulada >= 0.80) + 1

m_autovalor_1 = np.sum(autovalores > 1)

gk = np.array([
    (1 / p) * sum(1 / i for i in range(k, p + 1))
    for k in range(1, p + 1)
])

retiene_gk = prop_varianza > gk
m_gk = np.sum(retiene_gk)

autovalores_cattell = autovalores[autovalores > 1e-8]
componentes = np.arange(1, len(autovalores_cattell) + 1)

inicio_recta = 5

x_recta = componentes[inicio_recta - 1:]
y_recta = autovalores_cattell[inicio_recta - 1:]

coef = np.polyfit(x_recta, y_recta, deg=1)
recta = np.polyval(coef, componentes)

plt.figure(figsize=(8, 5))

plt.plot(
    componentes,
    autovalores_cattell,
    marker="o",
```

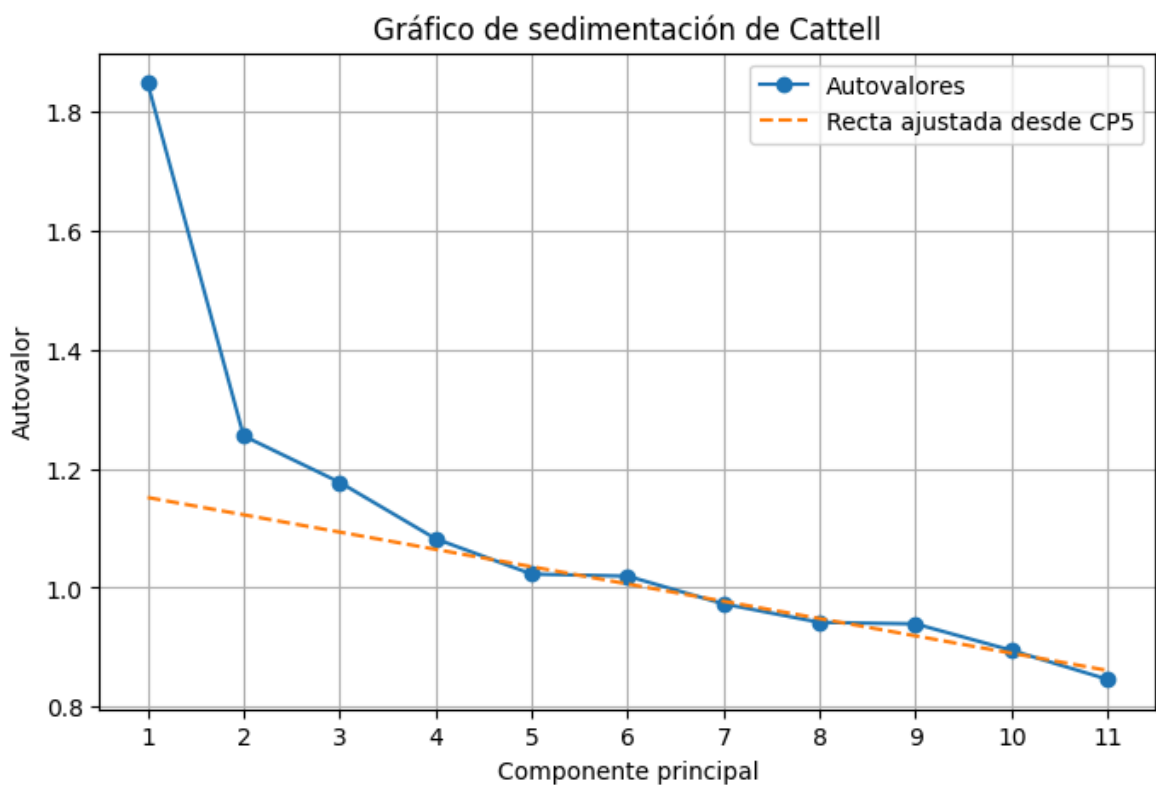
```

    label="Autovalores"
)

plt.plot(
    componentes,
    recta,
    linestyle="--",
    label=f"Recta ajustada desde CP{inicio_recta}"
)

plt.xlabel("Componente principal")
plt.ylabel("Autovalor")
plt.title("Gráfico de sedimentación de Cattell")
plt.xticks(componentes)
plt.grid(True)
plt.legend()
plt.show()

```



```

In [12]: resumen_retencion = pd.DataFrame({
    "Criterio": [
        "Varianza acumulada ≥ 65%",
        "Varianza acumulada ≥ 70%",
        "Varianza acumulada ≥ 75%",
        "Varianza acumulada ≥ 80%",
        "Kaiser: autovalor > 1",
        "Proporción mínima g_k",
        "Catell"
    ],
    "Número de componentes sugerido": [
        m_65,
        m_70,
        m_75,
        m_80,
        m_autovalor_1,
        m_gk,
        "3-4"
    ]
})

```

```

    ]
})

n_componentes = 8

cargas = pca.components_[:n_componentes].T * np.sqrt(pca.explained_variance_[:n_

matriz_cargas = pd.DataFrame(
    cargas,
    index=proporciones,
    columns=[f"CP{i+1}" for i in range(n_componentes)]
)

print("Resumen de criterios para selección de componentes principales")
display(resumen_retencion)
print("\nMatriz de cargas (hasta CP8)")
display(matriz_cargas.round(3))

```

Resumen de criterios para selección de componentes principales

	Criterio	Número de componentes sugerido
0	Varianza acumulada $\geq 65\%$	7
1	Varianza acumulada $\geq 70\%$	8
2	Varianza acumulada $\geq 75\%$	8
3	Varianza acumulada $\geq 80\%$	9
4	Kaiser: autovalor > 1	6
5	Proporción mínima g_k	7
6	Catell	3-4

Matriz de cargas (hasta CP8)

	CP1	CP2	CP3	CP4	CP5	CP6	CP7	CP8
prop_alimentacion	0.370	0.734	-0.146	-0.163	-0.037	-0.089	0.336	0.172
prop_alcohol_tabaco	0.003	0.305	-0.390	-0.509	-0.088	0.279	-0.164	-0.369
prop_ropa_calzado	-0.319	0.066	0.406	-0.222	-0.115	-0.097	0.020	0.337
prop_vivienda	0.775	-0.515	-0.032	0.085	0.222	0.030	0.028	0.032
prop_mobiliario	-0.018	0.360	0.361	0.150	0.598	-0.231	-0.308	-0.288
prop_salud	-0.076	0.198	0.116	0.610	-0.249	0.671	0.057	-0.086
prop_transporte	-0.532	-0.057	-0.517	0.322	-0.255	-0.466	-0.162	0.069
prop_comunicaciones	0.361	-0.243	0.070	-0.336	-0.350	0.029	-0.150	0.065
prop_ocio_cultura	-0.430	-0.016	0.171	-0.206	0.215	0.278	-0.009	0.542
prop_educacion	-0.265	-0.168	0.222	-0.065	-0.039	-0.168	0.774	-0.352
prop_hoteles_restaurantes	-0.573	-0.294	-0.017	-0.285	0.156	0.252	-0.089	-0.311
prop_bienes_servicios	0.079	0.090	0.586	-0.034	-0.521	-0.182	-0.278	-0.220

Se seleccionan 8 componentes principales basandose en los siguientes criterios:

- Se logra capturar el 77% de la varianza total. Si nos fijaásemos en el criterio de Kaiser, el del autovalor superior a 1, nos quedaríamos con 6, pero solo tendríamos un 61% de varianza explicado, un valor muy bajo si queremos interpretabilidad real y útil, y una buena predicción posterior, ya que al pasar a 8 estaríamos ganando un 16% de varianza explicada.

```
In [13]: def varimax_rotation(loadings, max_iter=1000, tol=1e-7):
    n_rows, n_cols = loadings.shape
    rotation_matrix = np.eye(n_cols)
    d = 0
    for i in range(max_iter):
        d_old = d
        transformed_loadings = np.dot(loadings, rotation_matrix)
        u, s, vh = np.linalg.svd(np.dot(loadings.T, transformed_loadings**3 -
                                         (1.0/n_rows) * np.dot(transformed_loadings,
                                                                np.diag(np.sum(transformed_loadings**2, axis=0))
                                         )
        rotation_matrix = np.dot(u, vh)
        d = np.sum(s)
        if d_old != 0 and d/d_old < 1 + tol: break
    return np.dot(loadings, rotation_matrix), rotation_matrix

n_componentes = 8
cargas_originales = pca.components_[:n_componentes].T * np.sqrt(pca.explained_var_)
cargas_rotadas, matriz_rotacion = varimax_rotation(cargas_originales)

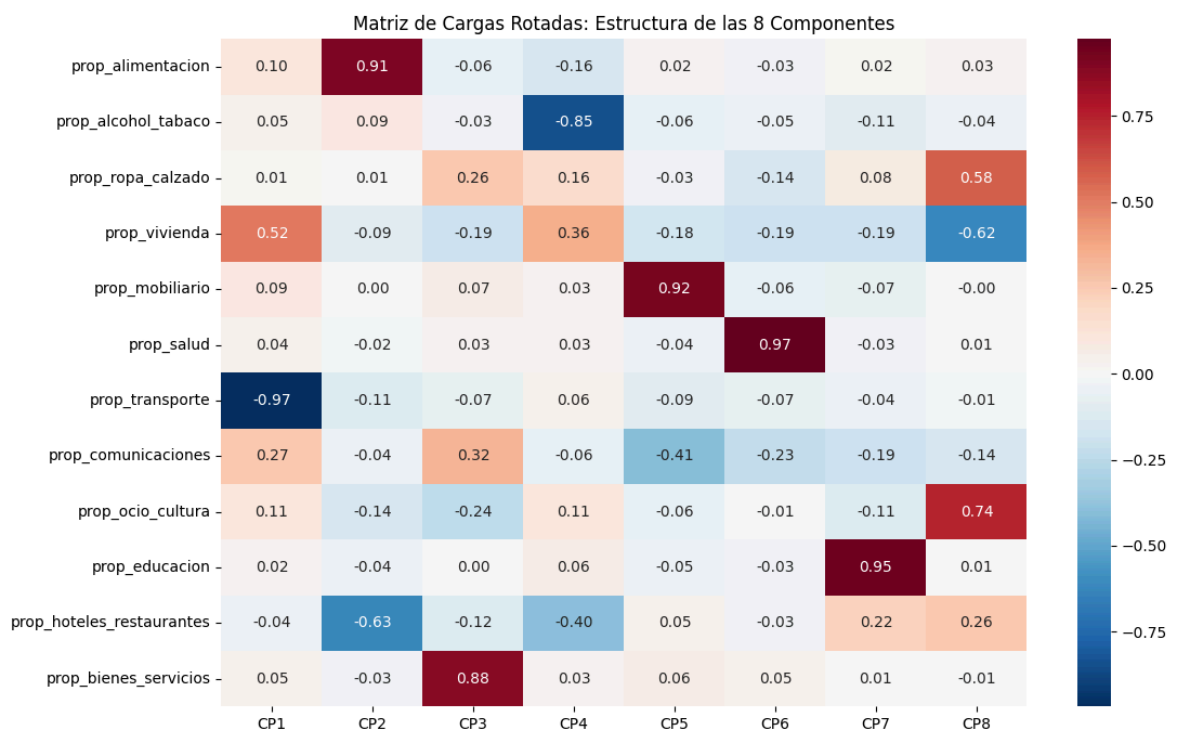
matriz_cargas_rotadas = pd.DataFrame(
    cargas_rotadas,
    index=proporciones,
    columns=[f"CP{i+1}" for i in range(n_componentes)]
)

print("Matriz de cargas tras rotación Varimax (hasta CP8)")
display(matriz_cargas_rotadas.round(3))
```

Matriz de cargas tras rotación Varimax (hasta CP8)

	CP1	CP2	CP3	CP4	CP5	CP6	CP7	CP8
prop_alimentacion	0.103	0.912	-0.063	-0.161	0.024	-0.028	0.017	0.027
prop_alcohol_tabaco	0.047	0.092	-0.032	-0.850	-0.060	-0.048	-0.105	-0.045
prop_ropa_calzado	0.012	0.007	0.261	0.161	-0.033	-0.136	0.079	0.580
prop_vivienda	0.517	-0.093	-0.189	0.356	-0.176	-0.188	-0.189	-0.621
prop_mobiliario	0.089	0.003	0.066	0.030	0.922	-0.063	-0.069	-0.005
prop_salud	0.041	-0.016	0.028	0.026	-0.042	0.974	-0.031	0.010
prop_transporte	-0.967	-0.111	-0.068	0.055	-0.088	-0.067	-0.043	-0.014
prop_comunicaciones	0.267	-0.039	0.325	-0.058	-0.411	-0.226	-0.191	-0.136
prop_ocio_cultura	0.113	-0.144	-0.239	0.106	-0.063	-0.007	-0.113	0.745
prop_educacion	0.021	-0.039	0.004	0.057	-0.052	-0.032	0.947	0.008
prop_hoteles_restaurantes	-0.035	-0.627	-0.125	-0.398	0.051	-0.028	0.219	0.264
prop_bienes_servicios	0.051	-0.028	0.883	0.032	0.062	0.046	0.007	-0.007

```
In [14]: plt.figure(figsize=(12, 8))
sns.heatmap(matriz_cargas_rotadas, annot=True, cmap='RdBu_r', center=0, fmt=".2f")
plt.title("Matriz de Cargas Rotadas: Estructura de las 8 Componentes")
plt.show()
```



```
In [15]: coordenadas_hogares_train = pca.transform(datos_prop_train_escalados)[: , :n_comp]
coordenadas_hogares_test = pca.transform(datos_prop_test_escalados)[: , :n_comp]

coordenadas_hogares_train_rot = np.dot(coordenadas_hogares_train, matriz_rotacion)
coordenadas_hogares_test_rot = np.dot(coordenadas_hogares_test, matriz_rotacion)

for i in range(1, 9):
```

```

datos_train[f"CP{i}"] = coordenadas_hogares_train_rot[:, i-1]
datos_test[f"CP{i}"] = coordenadas_hogares_test_rot[:, i-1]

```

```

In [16]: def plot_pca_complete_analysis(loadings, contribuciones, pca, pc_x, pc_y):
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

ax_biplot = axes[0]
for i, var in enumerate(loadings.index):
    var_clean = var.replace('prop_', '')

    x = loadings.iloc[i, pc_x-1]
    y = loadings.iloc[i, pc_y-1]

    var_rotada = np.var(coordenadas_hogares_train_rot, axis=0, ddof=1)
    var_pct_rotada = var_rotada / np.sum(pca.explained_variance_) * 100

    ax_biplot.arrow(0, 0, x, y, color='r', alpha=0.5, head_width=0.02)
    ax_biplot.text(x * 1.15, y * 1.15, var_clean, color='darkgreen',
                  ha='center', va='center', fontsize=9, fontweight='bold')

circle = plt.Circle((0,0), 1, color='navy', fill=False, linestyle='--')
ax_biplot.add_artist(circle)

ax_biplot.set_xlim(-1.2, 1.2)
ax_biplot.set_ylim(-1.2, 1.2)
ax_biplot.axhline(0, color='black', lw=1)
ax_biplot.axvline(0, color='black', lw=1)
ax_biplot.set_title(f'Mapa de Variables: CP{pc_x} vs CP{pc_y}')
ax_biplot.set_xlabel(f'CP{pc_x} ({var_pct_rotada[pc_x-1]:.2f}%)')
ax_biplot.set_ylabel(f'CP{pc_y} ({var_pct_rotada[pc_y-1]:.2f}%)')
ax_biplot.grid(True, alpha=0.3)

pcs_to_plot = [pc_x, pc_y]
mean_contribution = 100 / len(loadings)

for i, pc_idx in enumerate(pcs_to_plot):
    ax_bar = axes[i+1]
    cp_name = f"CP{pc_idx}"

    data_plot = contribuciones[cp_name].sort_values(ascending=False)
    data_plot.index = [idx.replace('prop_', '') for idx in data_plot.index]

    data_plot.plot(kind='bar', color='skyblue', ax=ax_bar)
    ax_bar.axhline(y=mean_contribution, color='red', linestyle='--', label='')

    ax_bar.set_title(f"Contribucion a la {cp_name}")
    ax_bar.set_ylabel("Contribucion (%)")
    ax_bar.tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

def plot_pca_biplot(ax, loadings, pca, pc_x, pc_y, coords_rot=None):

    if coords_rot is None:
        coords_rot = coordenadas_hogares_train_rot

    var_rotada = np.var(coords_rot, axis=0, ddof=1)
    var_pct_rotada = var_rotada / np.sum(pca.explained_variance_) * 100

```

```

for i, var in enumerate(loadings.index):
    var_clean = var.replace('prop_', '')
    x = loadings.iloc[i, pc_x-1]
    y = loadings.iloc[i, pc_y-1]
    ax.arrow(0, 0, x, y, color='r', alpha=0.5, head_width=0.02)
    ax.text(x * 1.15, y * 1.15, var_clean, color='darkgreen',
            ha='center', va='center', fontsize=9, fontweight='bold')

```

```

circle = plt.Circle((0, 0), 1, color='navy', fill=False, linestyle='--')
ax.add_artist(circle)
ax.set_xlim(-1.2, 1.2)
ax.set_ylim(-1.2, 1.2)
ax.axhline(0, color='black', lw=1)
ax.axvline(0, color='black', lw=1)
ax.set_title(f'CP{pc_x} vs CP{pc_y}')
ax.set_xlabel(f'CP{pc_x} ({var_pct_rotada[pc_x-1]:.2f}%)')
ax.set_ylabel(f'CP{pc_y} ({var_pct_rotada[pc_y-1]:.2f}%)')
ax.grid(True, alpha=0.3)

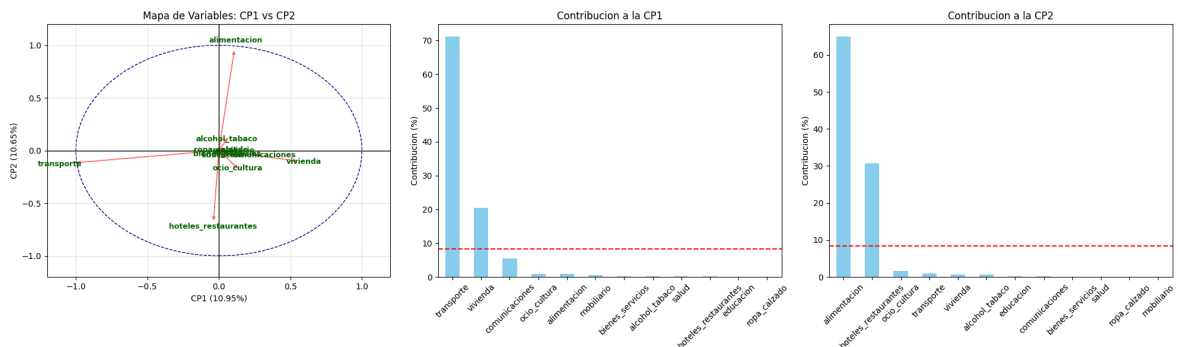
```

```

contribuciones = (matriz_cargas_rotadas**2) / (matriz_cargas_rotadas**2).sum(axes=1)

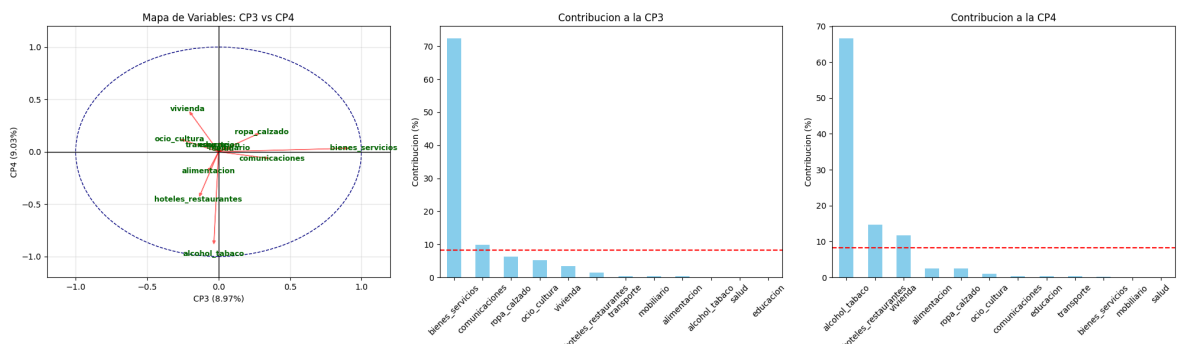
```

In [17]: `plot_pca_complete_analysis(matriz_cargas_rotadas, contribuciones, pca, 1, 2)`



- **CP1: Movilidad frente a Vivienda.** Diferencia los hogares con presupuesto en movilidad frente a los que tienen repartido su presupuesto en vivienda y comunicaciones.
- **CP2: Alimentación en casa frente a restauración.** Separa el gasto de "comer en casa" frente al estilo de vida de "comida fuera del hogar".

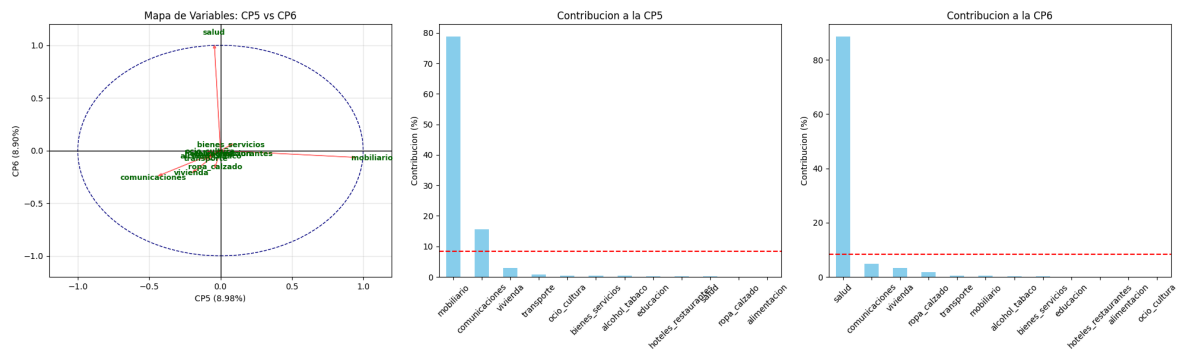
In [18]: `plot_pca_complete_analysis(matriz_cargas_rotadas, contribuciones, pca, 3, 4)`



- **CP3: Bienes y servicios personales.** Hogares con mucho consumo en bienes y servicios.

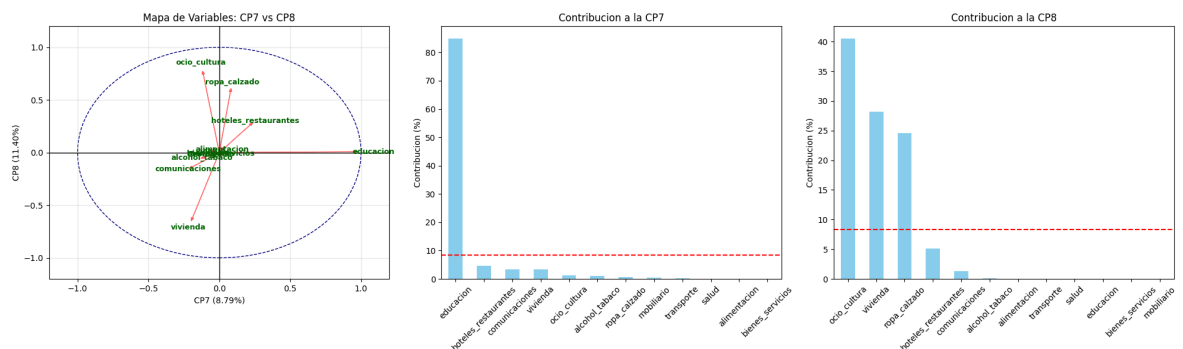
- **CP4:** *Vivienda frente a consumo en ropa, restauración y alcohol y tabaco.* Separa hogares con mayor peso en vivienda frente a hogares con más peso en ropa, alcohol y tabaco y hoteles-restaurantes.

```
In [19]: plot_pca_complete_analysis(matriz_cargas_rotadas, contribuciones, pca, 5, 6)
```



- **CP5:** *Equipamiento del hogar.* Hogares con peso alto en equipamiento del hogar frente a peso ligero en comunicaciones.
- **CP6:** *Salud.* Presupuesto destinado de forma clarísima a la salud.

```
In [20]: plot_pca_complete_analysis(matriz_cargas_rotadas, contribuciones, pca, 7, 8)
```



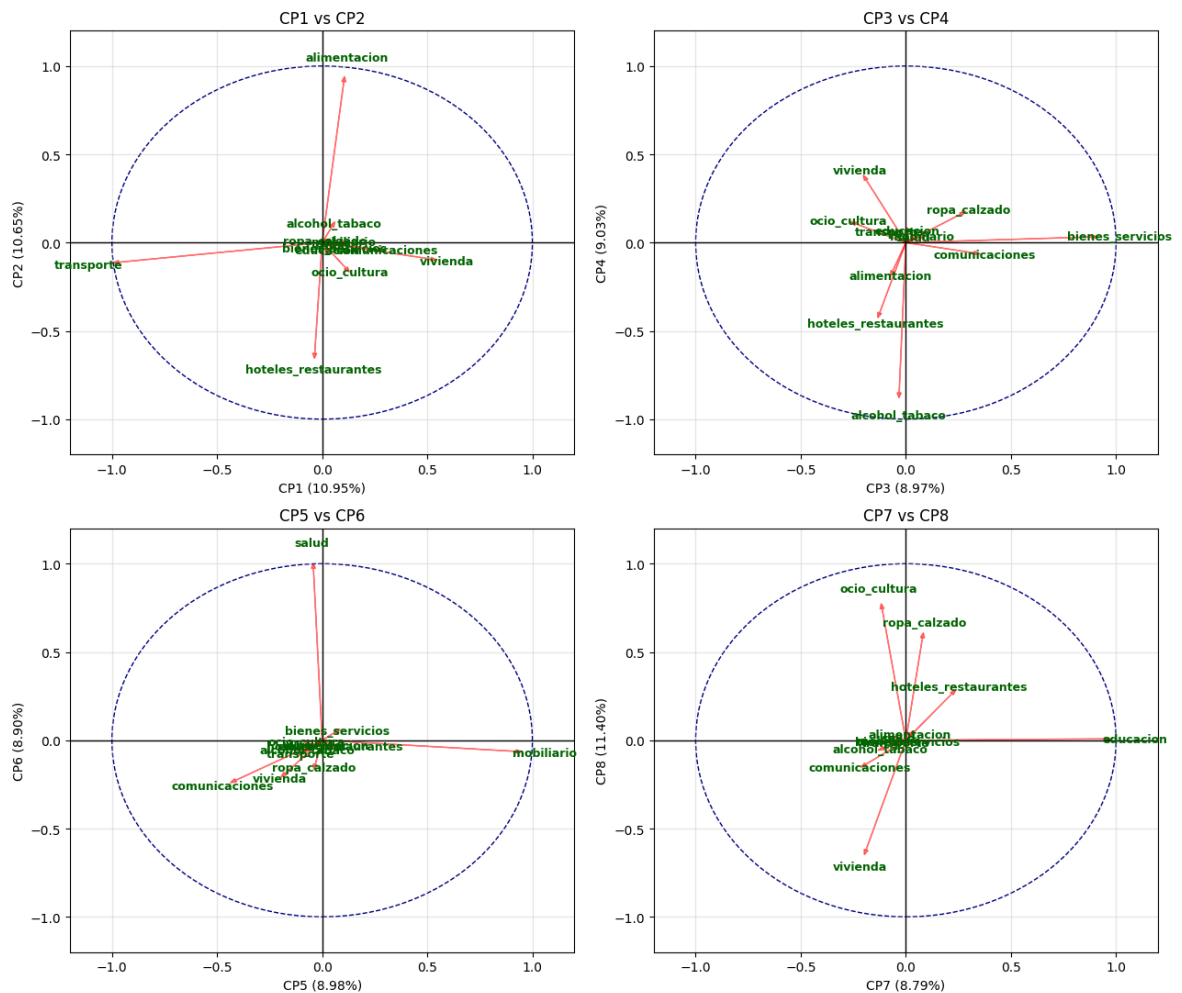
- **CP7:** *Educación.* Hogares con un mayor peso relativo en educación frente a hogares con mayor peso en alcohol y tabaco.
- **CP8:** *Ocio y cultura.* Enfrenta a hogares con un gran gasto en ocio y cultura y restauración, frente a hogares con un peso secundario de ropa, comunicaciones y vivienda.

```
In [21]: fig, axes = plt.subplots(2, 2, figsize=(13, 11))

pares = [(1, 2), (3, 4), (5, 6), (7, 8)]

for ax, (pc_x, pc_y) in zip(axes.ravel(), pares):
    plot_pca_biplot(ax, matriz_cargas_rotadas, pca, pc_x, pc_y, coordenadas_hoga

plt.tight_layout()
plt.show()
```



```
In [22]: cp_vars = {
    'CP1': 'movilidad_vs_vivienda',
    'CP2': 'alimentacion_vs_restauracion',
    'CP3': 'otros_bienes_servicios_personales',
    'CP4': 'vivienda_vs_restauracion_alcohol_tabaco',
    'CP5': 'equipamiento_hogar',
    'CP6': 'salud',
    'CP7': 'educacion',
    'CP8': 'ocio_imagen_personal_vs_vivienda',
}

datos_train = datos_train.rename(columns=cp_vars)
datos_test = datos_test.rename(columns=cp_vars)

cp_nombres = list(cp_vars.values())
```

Cluster

Selección del número de clusters

```
In [23]: k_values = range(2, 20)

resultados_k = []

X_cp_train = datos_train[cp_nombres].values
X_cp_test = datos_test[cp_nombres].values
```

```

for k in k_values:
    km = KMeans(
        n_clusters=k,
        n_init=100,
        random_state=111104
    )

    labels = km.fit_predict(X_cp_train)

    sizes = pd.Series(labels).value_counts(normalize=True).sort_index()

    resultados_k.append({
        "k": k,
        "Inercia": km.inertia_,
        "Silueta": silhouette_score(X_cp_train, labels),
        "Pseudo F": calinski_harabasz_score(X_cp_train, labels),
        "Tamaño cluster más pequeño (%)": sizes.min() * 100,
        "Tamaño cluster más grande (%)": sizes.max() * 100
    })

metricas_cluster = pd.DataFrame(resultados_k)

display(metricas_cluster)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].plot(metricas_cluster["k"], metricas_cluster["Inercia"], marker="o", color="red")
axes[0].set_title("Inercia")
axes[0].set_xlabel("Número de clusters k")
axes[0].set_xticks(metricas_cluster["k"])

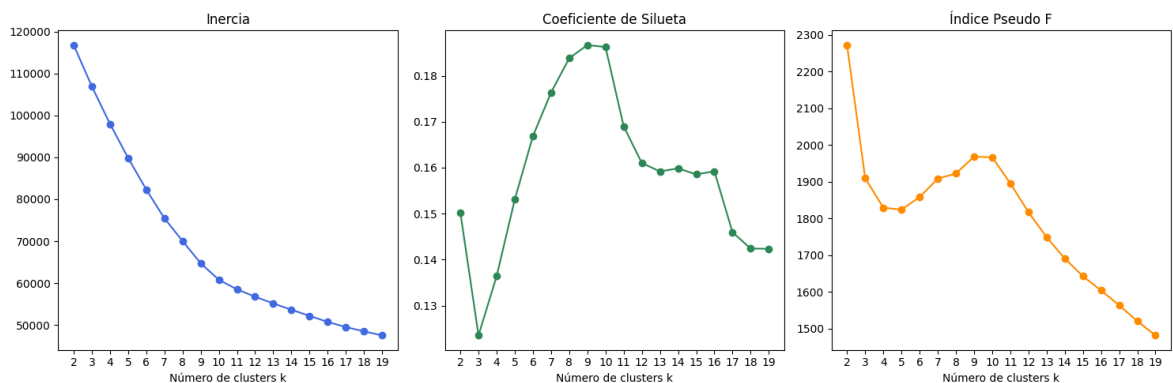
axes[1].plot(metricas_cluster["k"], metricas_cluster["Silueta"], marker="o", color="blue")
axes[1].set_title("Coeficiente de Silueta")
axes[1].set_xlabel("Número de clusters k")
axes[1].set_xticks(metricas_cluster["k"])

axes[2].plot(metricas_cluster["k"], metricas_cluster["Pseudo F"], marker="o", color="green")
axes[2].set_title("Índice Pseudo F")
axes[2].set_xlabel("Número de clusters k")
axes[2].set_xticks(metricas_cluster["k"])

plt.tight_layout()
plt.show()

```

	k	Inercia	Silueta	Pseudo F	Tamaño cluster más pequeño (%)	Tamaño cluster más grande (%)
0	2	116764.136951	0.150205	2271.655859	47.788588	52.211412
1	3	106900.616171	0.123523	1909.034410	27.668530	37.431864
2	4	97966.585700	0.136425	1829.087282	11.902298	32.912440
3	5	89838.231459	0.153097	1823.546773	7.182778	28.572414
4	6	82297.401700	0.166877	1857.880799	3.822535	27.716829
5	7	75446.319660	0.176351	1907.947322	3.698337	23.935693
6	8	70027.479586	0.183797	1921.942904	3.463741	21.755330
7	9	64720.146381	0.186663	1967.941864	3.353343	20.906645
8	10	60792.379803	0.186235	1966.145678	2.973849	20.209756
9	11	58503.186472	0.169003	1895.314733	3.111847	18.581384
10	12	56753.580447	0.161023	1816.589698	3.084248	17.808597
11	13	55150.036388	0.159174	1748.594664	2.877251	14.503553
12	14	53641.521366	0.159856	1690.687218	1.138481	13.344373
13	15	52186.793571	0.158535	1642.398380	1.138481	12.405989
14	16	50743.045646	0.159213	1603.871406	0.903885	12.033395
15	17	49512.970001	0.145955	1563.353720	0.883185	11.239909
16	18	48493.565725	0.142413	1520.121202	0.896985	9.825433
17	19	47522.594088	0.142349	1481.336958	0.876285	9.894432



Para determinar el número óptimo de tipologías (k), se evalúan particiones desde 2 hasta 19.

- El estadístico de la pseudo F se mantiene elevado en todas las particiones, y en especial en $k = 2$ y $k = 9$
- El coeficiente de silueta presenta sus únicos máximos locales en $k = 2$ y $k = 9$.
- En la inercia parece que se estabiliza ligeramente entre $k = 10$

Nos quedaremos en $k = 9$, ya que las medidas son muy parecidas entre 9 y 10, y coger un grupo más sería meter ruido y sobreajustar el futuro modelo.

Interpretación de los clusters

```
In [24]: def resumen_clusters_k(X_scaled_df, df, cp_vars, k, random_state=111104):
    km = KMeans(
        n_clusters=k,
        n_init=100,
        random_state=random_state
    )

    labels = km.fit_predict(X_scaled_df)

    col_cluster = f"cluster_k{k}"
    if hasattr(X_scaled_df, "index"):
        idx = X_scaled_df.index
    else:
        idx = df.index
    df.loc[idx, col_cluster] = labels

    centroides = pd.DataFrame(
        km.cluster_centers_,
        columns=cp_vars
    )

    centroides.index = [f"Cluster {i}" for i in range(k)]

    tamanos = (
        pd.Series(labels)
        .value_counts()
        .sort_index()
        .to_frame("n")
    )
    tamanos["%"] = 100 * tamanos["n"] / tamanos["n"].sum()
    tamanos.index = [f"Cluster {i}" for i in range(k)]

    return centroides.round(3), tamanos.round(2), labels

centroides_9, tamanos_9, labels_9 = resumen_clusters_k(
    X_cp_train, datos_train, cp_vars.values(), k=9
)

kmeans_final = KMeans(n_clusters=9, n_init=100, random_state=111104)
datos_train['cluster_9'] = kmeans_final.fit_predict(X_cp_train)
datos_test['cluster_9'] = kmeans_final.predict(X_cp_test)
columnas_gasto = proporciones
```

```

perfil_gasto = datos_train.groupby('cluster_9')[columnas_gasto].mean() * 100

nombres_clusters = {
    0: "Ocio y restauración",
    1: "Movilidad intensiva",
    2: "Educación",
    3: "Alcohol y tabaco",
    4: "Vivienda",
    5: "Bienes, servicios e imagen personal",
    6: "Equipamiento del hogar",
    7: "Consumo básico doméstico",
    8: "Salud",
}

datos_train["perfil_consumo"] = datos_train["cluster_9"].map(nombres_clusters)
datos_test["perfil_consumo"] = datos_test["cluster_9"].map(nombres_clusters)

print("Tamaños k=9")
display(tamanos_9.transpose())
print("\nCentroides k=9")
display(centroides_9)
print("\nPerfil de gasto k=9")
display(perfil_gasto.round(2))

```

Tamaños k=9

	Cluster 0	Cluster 1	Cluster 2	Cluster 3	Cluster 4	Cluster 5	Cluster 6	Cluster 7	Cluster 8
n	3030.00	1520.00	486.00	759.00	2839.00	1411.00	519.00	3030.00	899.0
%	20.91	10.49	3.35	5.24	19.59	9.74	3.58	20.91	6.2

Centroides k=9

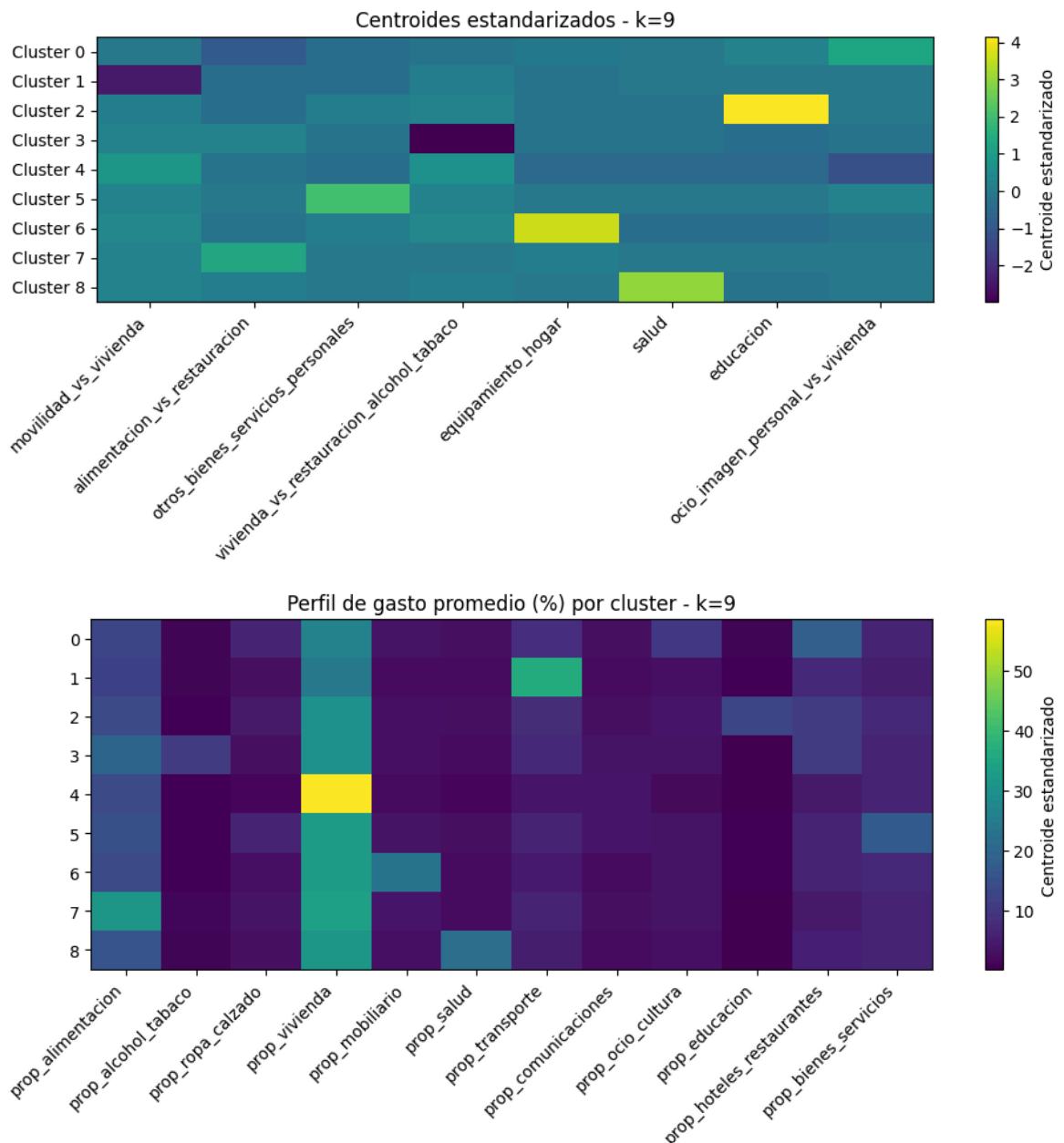
	movilidad_vs_vivienda	alimentacion_vs_restauracion	otros_bienes_servicios_personales
Cluster 0	-0.058	-0.918	-0.000
Cluster 1	-2.467	-0.366	-0.000
Cluster 2	0.079	-0.342	0.000
Cluster 3	0.192	0.289	-0.000
Cluster 4	0.880	-0.206	-0.000
Cluster 5	0.223	-0.113	2.000
Cluster 6	0.373	-0.185	0.000
Cluster 7	0.184	1.347	-0.000
Cluster 8	0.197	0.048	-0.000

◀ Perfil de gasto k=9 ▶

	prop_alimentacion	prop_alcohol_tabaco	prop_ropa_calzado	prop_vivienda	prop_otros_bienes_servicios_personales
cluster_9					
0	12.77	0.99	6.00	26.91	0.00
1	12.07	0.98	2.87	23.66	0.00
2	13.66	0.71	4.15	29.85	0.00
3	19.85	11.35	2.68	29.78	0.00
4	14.33	0.67	1.76	58.51	0.00
5	15.30	0.77	6.65	32.32	0.00
6	13.88	0.78	3.18	32.50	0.00
7	32.00	1.31	3.44	33.69	0.00
8	16.36	1.03	2.74	31.04	0.00

```
In [25]: def plot_centroides(centroides, titulo):
plt.figure(figsize=(10, 5))
plt.imshow(centroides, aspect="auto")
plt.colorbar(label="Centroide estandarizado")
plt.xticks(range(len(centroides.columns)), centroides.columns, rotation=45,
plt.yticks(range(len(centroides.index)), centroides.index)
plt.title(titulo)
plt.tight_layout()
plt.show()
```

```
plot_centroides(centroides_9, "Centroides estandarizados - k=9")
plot_centroides(perfil_gasto, "Perfil de gasto promedio (%) por cluster - k=9")
```



Caracterización sociodemográfica

```
In [26]: cluster_col = "perfil_consumo"
datos_caracterizacion = datos_train.copy()

suple_numericas = [
    "tamano_ocde", "n_menores", "n_mayores", "n_ocupados",
    "log_ingresos_hogar", "n_habitaciones", "log_gasto_total_crudo"
]

suple_categoricas = [
    "c_autonoma", "tam_municipio", "hombre",
    "ocupado", "estudios", "sector_publico_imp", "alquiler",
    "tipo_hogar", "edad"
]

categoricas_graficar = [
```

```

"tipo_hogar", "edad", "ocupado", "estudios", "tam_municipio"
]

orden_clusters = sorted(datos_caracterizacion[cluster_col].dropna().unique())

media_global = datos_caracterizacion[proporciones].mean() * 100
desv_gasto = perfil_gasto - media_global

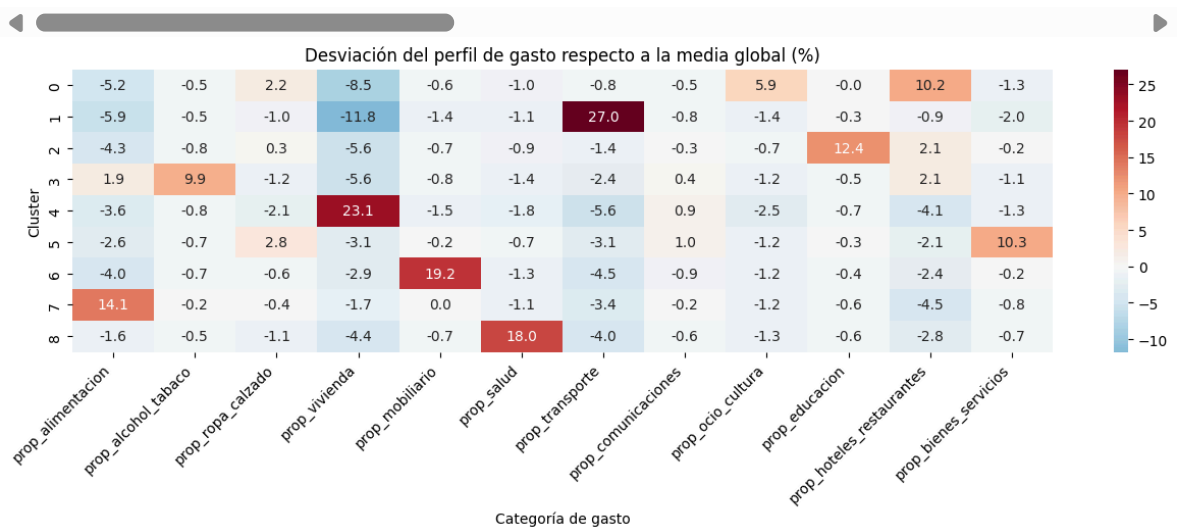
print("Desviación respecto a la media global (puntos porcentuales)")
display(desv_gasto.round(2))

plt.figure(figsize=(13, 5))
sns.heatmap(desv_gasto, cmap="RdBu_r", center=0, annot=True, fmt=".1f")
plt.title("Desviación del perfil de gasto respecto a la media global (%)")
plt.xlabel("Categoría de gasto")
plt.ylabel("Cluster")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```

Desviación respecto a la media global (puntos porcentuales)

	prop_alimentacion	prop_alcohol_tabaco	prop_ropa_calzado	prop_vivienda	prop
cluster_9					
0	-5.16	-0.51	2.17	-8.49	
1	-5.86	-0.52	-0.96	-11.75	
2	-4.27	-0.78	0.32	-5.55	
3	1.92	9.85	-1.15	-5.62	
4	-3.60	-0.83	-2.07	23.10	
5	-2.63	-0.73	2.82	-3.09	
6	-4.05	-0.72	-0.65	-2.91	
7	14.07	-0.19	-0.39	-1.72	
8	-1.57	-0.47	-1.09	-4.37	



In [27]: `perfil_gasto = datos_caracterizacion.groupby(cluster_col)[proporciones].mean().1`

```

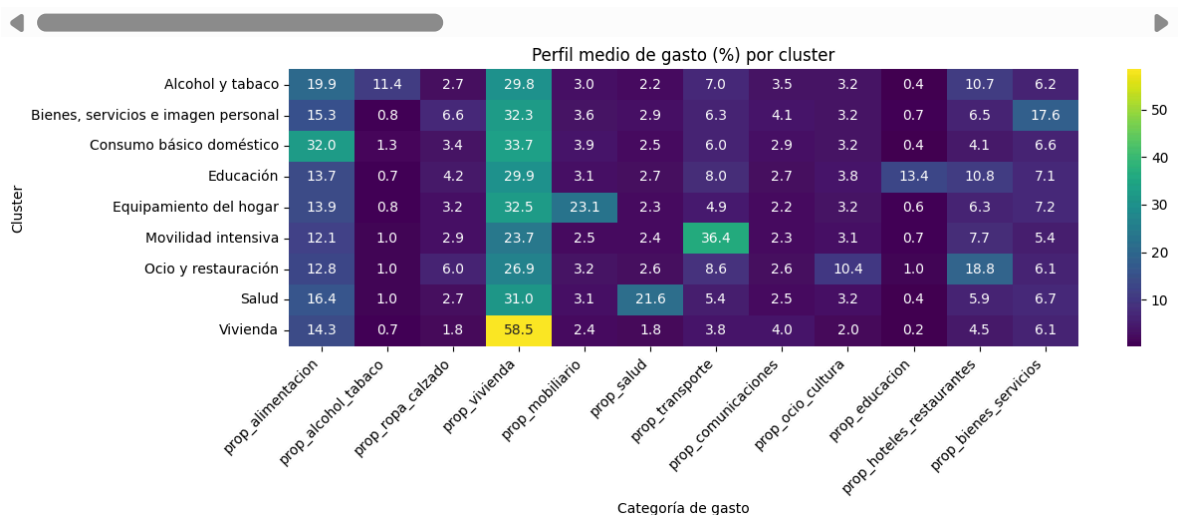
print("Perfil medio de gasto (%) por cluster")
display(perfil_gasto.round(2))

plt.figure(figsize=(13, 5))
sns.heatmap(perfil_gasto, cmap="viridis", annot=True, fmt=".1f")
plt.title("Perfil medio de gasto (%) por cluster")
plt.xlabel("Categoría de gasto")
plt.ylabel("Cluster")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```

Perfil medio de gasto (%) por cluster

	prop_alimentacion	prop_alcohol_tabaco	prop_ropa_calzado	prop_vivienda
perfil_consumo				
Alcohol y tabaco	19.85	11.35	2.68	29.76
Bienes, servicios e imagen personal	15.30	0.77	6.65	32.32
Consumo básico doméstico	32.00	1.31	3.44	33.69
Educación	13.66	0.71	4.15	29.85
Equipamiento del hogar	13.88	0.78	3.18	32.50
Movilidad intensiva	12.07	0.98	2.87	23.66
Ocio y restauración	12.77	0.99	6.00	26.91
Salud	16.36	1.03	2.74	31.04
Vivienda	14.33	0.67	1.76	58.51



In [28]: fig, axes = plt.subplots(1, 4, figsize=(20, 5))

```

sns.scatterplot(data=datos_train, x='movilidad_vs_vivienda', y='alimentacion_vs_
                ax=axes[0], palette='Set1', alpha=0.4, edgecolor=None)

sns.scatterplot(data=datos_train, x='otros_bienes_servicios_personales', y='vivi
                ax=axes[1], palette='Set1', alpha=0.4, edgecolor=None)

sns.scatterplot(data=datos_train, x='equipamiento_hogar', y='salud', hue='perfil
                ax=axes[2], palette='Set1', alpha=0.4, edgecolor=None)

sns.scatterplot(data=datos_train, x='educacion', y='ocio_imagen_personal_vs_vivi
                ax=axes[3], palette='Set1', alpha=0.4, edgecolor=None)

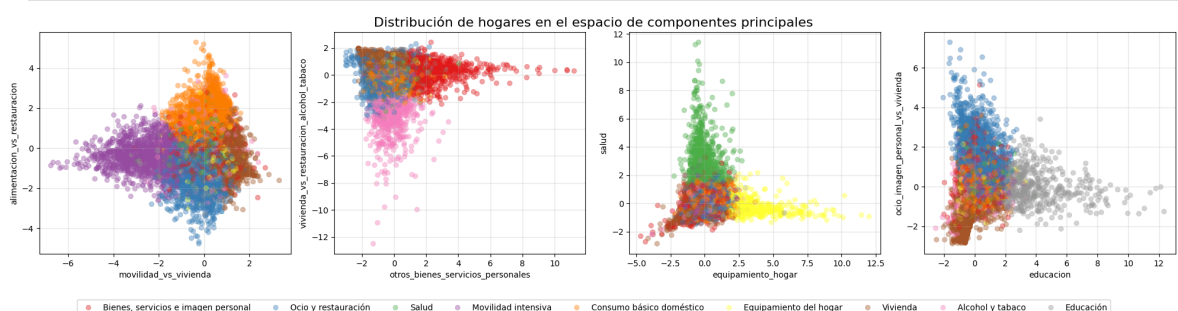
handles, labels = axes[0].get_legend_handles_labels()
labels_formateadas = []
for label in labels:
    try:
        cluster_id = int(float(label))
        nombre = nombres_clusters.get(cluster_id, str(cluster_id))
        labels_formateadas.append(f"{cluster_id}: {nombre}")
    except ValueError:
        labels_formateadas.append(label)

for ax in axes:
    ax.legend().remove()
    ax.grid(True, alpha=0.3)

fig.legend(
    handles,
    labels_formateadas,
    loc='lower center',
    ncol=9,
    bbox_to_anchor=(0.5, 0)
)

plt.tight_layout()
plt.subplots_adjust(bottom=0.22)
plt.suptitle("Distribución de hogares en el espacio de componentes principales",
plt.show()

```



```

In [29]: def cramers_v(tabla, corregida=True):
    chi2, p, dof, expected = chi2_contingency(tabla)
    n = tabla.to_numpy().sum()
    r, k = tabla.shape

    if not corregida:
        v = np.sqrt((chi2 / n) / min(r - 1, k - 1))
        return v, p

    phi2 = chi2 / n
    phi2_corr = max(0, phi2 - ((k - 1) * (r - 1)) / (n - 1))

```

```

r_corr = r - ((r - 1) ** 2) / (n - 1)
k_corr = k - ((k - 1) ** 2) / (n - 1)

v_corr = np.sqrt(phi2_corr / min(k_corr - 1, r_corr - 1))
return v_corr, p

def residuos_corregidos(tabla):
    chi2, p, dof, expected = chi2_contingency(tabla)
    expected = pd.DataFrame(expected, index=tabla.index, columns=tabla.columns)

    n = tabla.to_numpy().sum()
    prop_filas = tabla.sum(axis=1) / n
    prop_cols = tabla.sum(axis=0) / n

    residuos = pd.DataFrame(index=tabla.index, columns=tabla.columns, dtype=float)

    for i in tabla.index:
        for j in tabla.columns:
            residuos.loc[i, j] = (
                (tabla.loc[i, j] - expected.loc[i, j]) /
                np.sqrt(
                    expected.loc[i, j]
                    * (1 - prop_filas.loc[i])
                    * (1 - prop_cols.loc[j])
                )
            )

    return residuos

```

```

In [30]: tabla_tipo_hogar = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["tipo_hogar"]
)

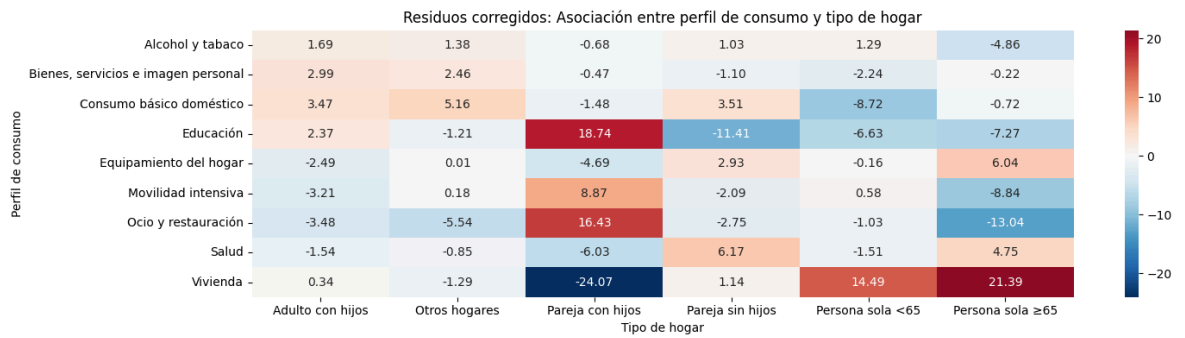
v_tipo_hogar, p_tipo_hogar = cramers_v(tabla_tipo_hogar)
res_tipo_hogar = residuos_corregidos(tabla_tipo_hogar)

print("V de Cramer:", round(v_tipo_hogar, 3))
print("p-valor chi-cuadrado:", p_tipo_hogar)

plt.figure(figsize=(15, max(3, 0.45 * res_tipo_hogar.shape[0])))
sns.heatmap(res_tipo_hogar, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y tipo de hogar")
plt.xlabel("Tipo de hogar")
plt.ylabel("Perfil de consumo")
plt.tight_layout()
plt.show()

```

V de Cramer: 0.159
p-valor chi-cuadrado: 0.0



```
In [31]: tabla_edad = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["edad"]
)

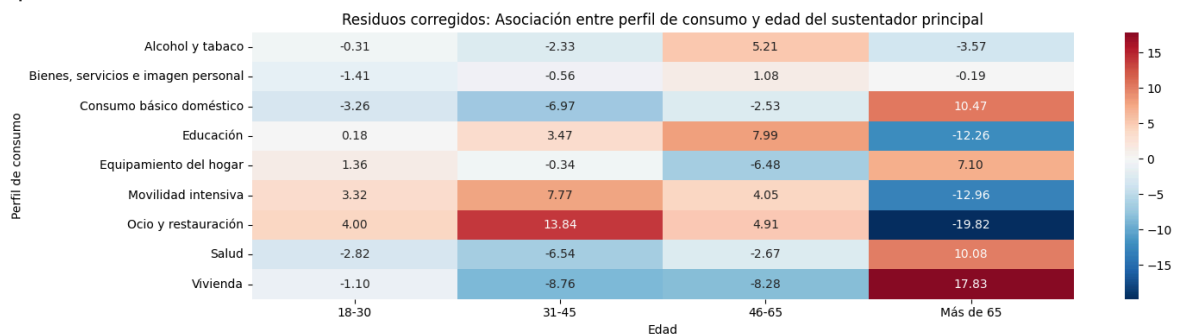
v_edad, p_edad = cramers_v(tabla_edad)
res_edad = residuos_corregidos(tabla_edad)

print("V de Cramer:", round(v_edad, 3))
print("p-valor chi-cuadrado:", p_edad)

plt.figure(figsize=(15, max(3, 0.45 * res_edad.shape[0])))
sns.heatmap(res_edad, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y edad del su")
plt.xlabel("Edad")
plt.ylabel("Perfil de consumo")
plt.tight_layout()
plt.show()
```

V de Cramer: 0.168

p-valor chi-cuadrado: 1.3899865236864994e-248



```
In [32]: tabla_estudios = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["estudios"]
)

v_estudios, p_estudios = cramers_v(tabla_estudios)
res_estudios = residuos_corregidos(tabla_estudios)

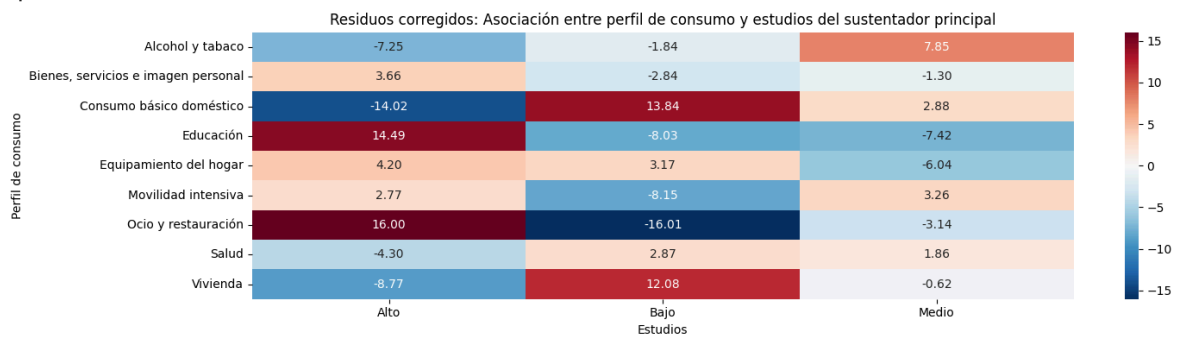
print("V de Cramer:", round(v_estudios, 3))
print("p-valor chi-cuadrado:", p_estudios)

plt.figure(figsize=(15, max(3, 0.45 * res_estudios.shape[0])))
sns.heatmap(res_estudios, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y estudios de")
plt.xlabel("Estudios")
plt.ylabel("Perfil de consumo")
```

```
plt.tight_layout()
plt.show()
```

V de Cramer: 0.196

p-valor chi-cuadrado: 1.6242413152826554e-231



```
In [33]: tabla_hombre = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["hombre"]
)

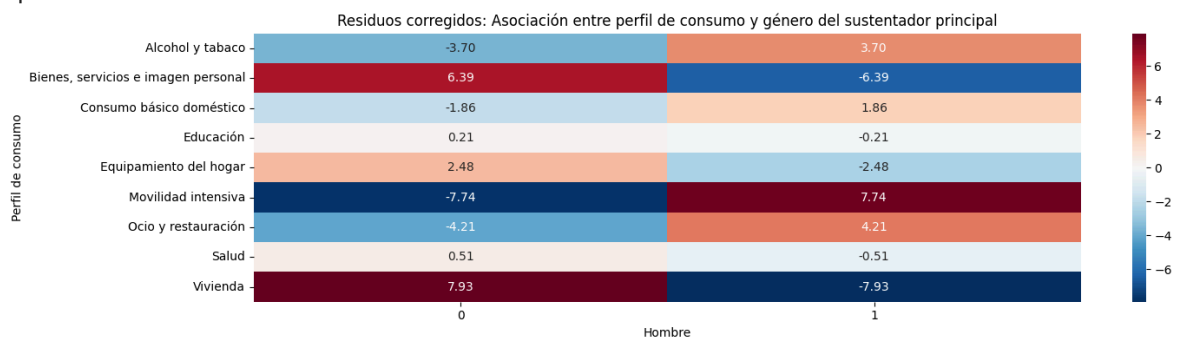
v_hombre, p_hombre = cramers_v(tabla_hombre)
res_hombre = residuos_corregidos(tabla_hombre)

print("V de Cramer:", round(v_hombre, 3))
print("p-valor chi-cuadrado:", p_hombre)

plt.figure(figsize=(15, max(3, 0.45 * res_hombre.shape[0])))
sns.heatmap(res_hombre, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y género del")
plt.xlabel("Hombre")
plt.ylabel("Perfil de consumo")
plt.tight_layout()
plt.show()
```

V de Cramer: 0.108

p-valor chi-cuadrado: 4.618988816337891e-34



```
In [34]: tabla_ocupado = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["ocupado"]
)

v_ocupado, p_ocupado = cramers_v(tabla_ocupado)
res_ocupado = residuos_corregidos(tabla_ocupado)

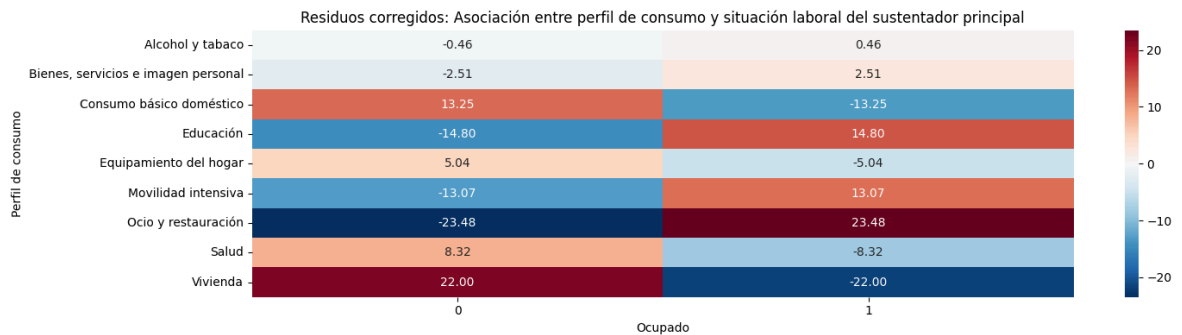
print("V de Cramer:", round(v_ocupado, 3))
print("p-valor chi-cuadrado:", p_ocupado)

plt.figure(figsize=(15, max(3, 0.45 * res_ocupado.shape[0])))
```

```
sns.heatmap(res_ocupado, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y situación l
plt.xlabel("Ocupado")
plt.ylabel("Perfil de consumo")
plt.tight_layout()
plt.show()
```

V de Cramer: 0.313

p-valor chi-cuadrado: 3.3970320091712247e-302



```
In [35]: tabla_sector_publico = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["sector_publico_imp"]
)

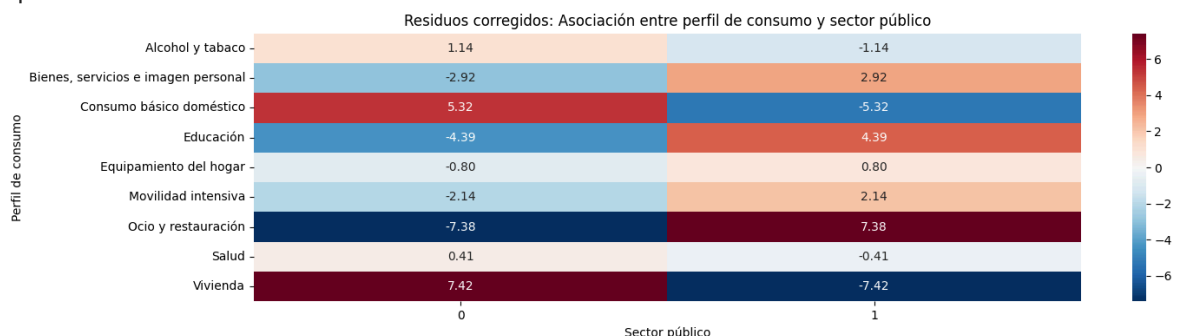
v_sector_publico, p_sector_publico = cramers_v(tabla_sector_publico)
res_sector_publico = residuos_corregidos(tabla_sector_publico)

print("V de Cramer:", round(v_sector_publico, 3))
print("p-valor chi-cuadrado:", p_sector_publico)

plt.figure(figsize=(15, max(3, 0.45 * res_sector_publico.shape[0])))
sns.heatmap(res_sector_publico, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y sector públ
plt.xlabel("Sector público")
plt.ylabel("Perfil de consumo")
plt.tight_layout()
plt.show()
```

V de Cramer: 0.096

p-valor chi-cuadrado: 8.204840597615395e-27



```
In [36]: tabla_alquiler = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["alquiler"]
)

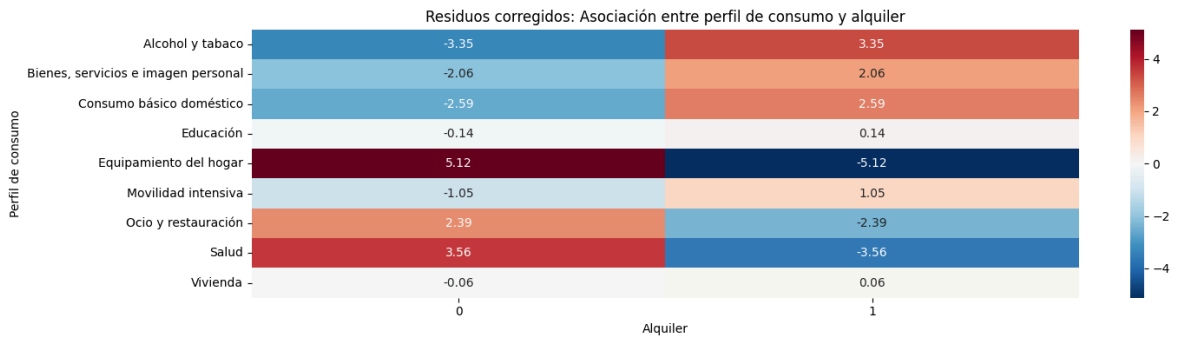
v_alquiler, p_alquiler = cramers_v(tabla_alquiler)
res_alquiler = residuos_corregidos(tabla_alquiler)
```

```
print("V de Cramer:", round(v_alquiler, 3))
print("p-valor chi-cuadrado:", p_alquiler)

plt.figure(figsize=(15, max(3, 0.45 * res_alquiler.shape[0])))
sns.heatmap(res_alquiler, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y alquiler")
plt.xlabel("Alquiler")
plt.ylabel("Perfil de consumo")
plt.tight_layout()
plt.show()
```

V de Cramer: 0.061

p-valor chi-cuadrado: 1.5509453131186202e-10



```
In [37]: tabla_tam_municipio = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["tam_municipio"]
)

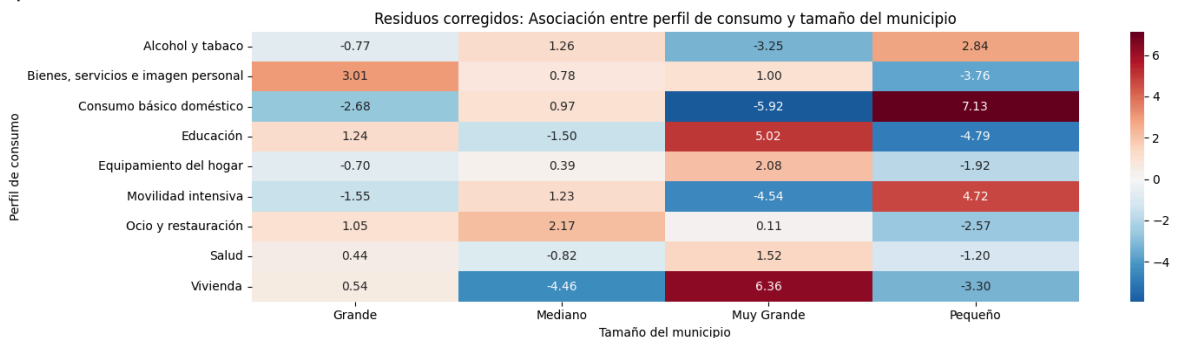
v_tam_municipio, p_tam_municipio = cramers_v(tabla_tam_municipio)
res_tam_municipio = residuos_corregidos(tabla_tam_municipio)

print("V de Cramer:", round(v_tam_municipio, 3))
print("p-valor chi-cuadrado:", p_tam_municipio)

plt.figure(figsize=(15, max(3, 0.45 * res_tam_municipio.shape[0])))
sns.heatmap(res_tam_municipio, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y tamaño del municipio")
plt.xlabel("Tamaño del municipio")
plt.ylabel("Perfil de consumo")
plt.tight_layout()
plt.show()
```

V de Cramer: 0.063

p-valor chi-cuadrado: 5.147839685569431e-29



```
In [38]: tabla_c_autonoma = pd.crosstab(
    datos_caracterizacion[cluster_col],
    datos_caracterizacion["c_autonoma"]
)
```

```

v_c_autonoma, p_c_autonoma = cramers_v(tabla_c_autonoma)
res_c_autonoma = residuos_corregidos(tabla_c_autonoma)

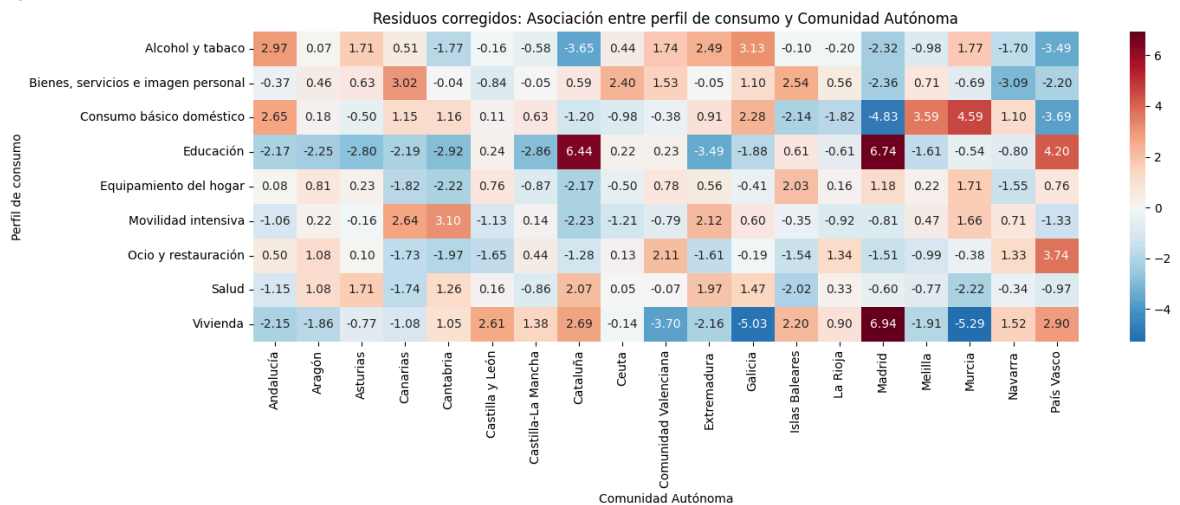
print("V de Cramer:", round(v_c_autonoma, 3))
print("p-valor chi-cuadrado:", p_c_autonoma)

plt.figure(figsize=(15, max(6, 0.45 * res_c_autonoma.shape[0])))
sns.heatmap(res_c_autonoma, cmap="RdBu_r", center=0, annot=True, fmt=".2f")
plt.title("Residuos corregidos: Asociación entre perfil de consumo y Comunidad A")
plt.xlabel("Comunidad Autónoma")
plt.ylabel("Perfil de consumo")
plt.tight_layout()
plt.show()

```

V de Cramer: 0.06

p-valor chi-cuadrado: 1.1105805347100178e-51



```

In [39]: resultados_kw = []

for var in suple_numericas:
    df_var = datos_caracterizacion[[cluster_col, var]].dropna()

    grupos = [
        df_var.loc[df_var[cluster_col] == cl, var].values
        for cl in orden_clusters
        if (df_var[cluster_col] == cl).sum() > 0
    ]

    H, p = kruskal(*grupos)

    n = len(df_var)
    k = len(grupos)

    epsilon2 = (H - k + 1) / (n - k)
    epsilon2 = max(0, epsilon2)

    resultados_kw.append({
        "Variable": var,
        "p-valor": p,
        "epsilon2": epsilon2
    })

tabla_kw = pd.DataFrame(resultados_kw)

```

```

tabla_kw["p-valor_formato"] = tabla_kw["p-valor"].apply(
    lambda x: "$<0.001$" if x < 0.001 else f"{x:.3f}"
)

tabla_kw = tabla_kw[
    ["Variable", "p-valor_formato", "epsilon2"]
].sort_values("epsilon2", ascending=False)

print("Contraste de Kruskal-Wallis y tamaño de efecto")
display(tabla_kw)

```

Contraste de Kruskal-Wallis y tamaño de efecto

	Variable	p-valor_formato	epsilon2
6	log_gasto_total_crudo	< 0.001	0.245449
3	n_ocupados	< 0.001	0.128427
4	log_ingresos_hogar	< 0.001	0.117320
0	tamano_ocde	< 0.001	0.091830
2	n_mayores	< 0.001	0.056722
1	n_menores	< 0.001	0.056604
5	n_habitaciones	< 0.001	0.002139

```

In [40]: suple_numericas_presentes = [
    var for var in suple_numericas
    if var in datos_caracterizacion.columns
]
faltantes = sorted(set(suple_numericas) - set(suple_numericas_presentes))
if faltantes:
    print("Variables no encontradas y omitidas:", ", ".join(faltantes))

resumen_num = (
    datos_caracterizacion
    .groupby(cluster_col)[suple_numericas_presentes]
    .mean()
    .loc[orden_clusters]
)

media_global = datos_caracterizacion[suple_numericas_presentes].mean()
sd_global = datos_caracterizacion[suple_numericas_presentes].std()

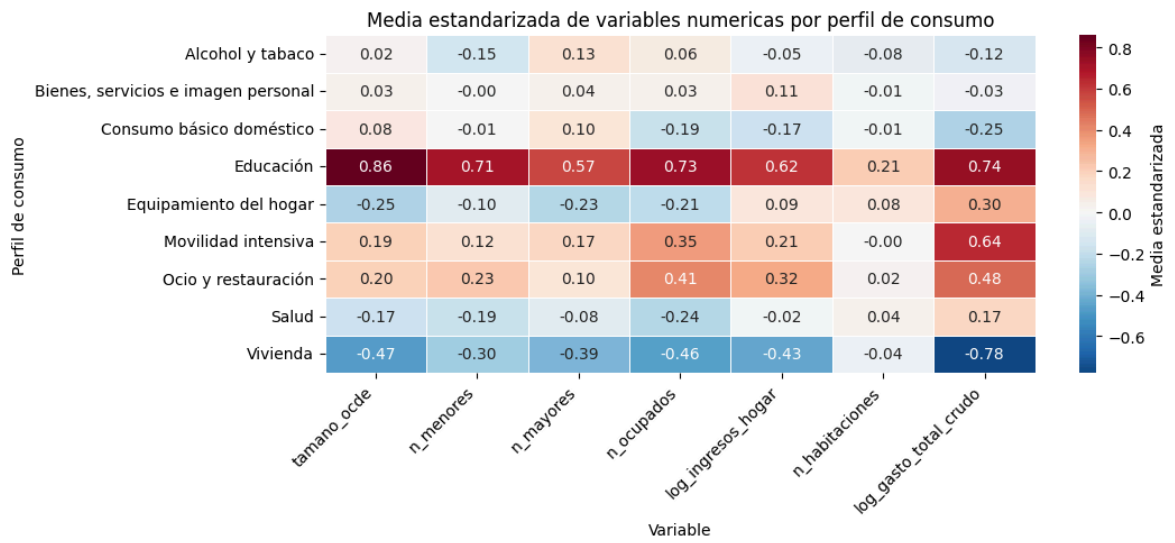
resumen_num_std_plot = (resumen_num - media_global) / sd_global

plt.figure(figsize=(11, 5))
sns.heatmap(
    resumen_num_std_plot,
    cmap="RdBu_r",
    center=0,
    annot=True,
    fmt=".2f",
    linewidths=0.5,
    cbar_kws={"label": "Media estandarizada"}
)

plt.title("Media estandarizada de variables numericas por perfil de consumo")

```

```
plt.xlabel("Variable")
plt.ylabel("Perfil de consumo")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```



```
In [41]: datos_modelos = pd.concat(
    [datos_train, datos_test],
    axis=0
)

datos_modelos.to_csv(
    "../datos_procesados/datos_con_tipologia_consumo.csv",
    index=False
)
```